



# Семинар 7.10: Введение в NLP

Курс: Архитектор в области искусственного интеллекта

Поляк Марк Дмитриевич

20 июля 2026

# Обзор семинара

## Формат

2 занятия: 20.07 (от текста к вектору) и 22.07 (эмбединги, классификация, взгляд архитектора)

### Занятие 1:

- Предобработка текста
- Bag of Words, TF-IDF
- N-граммы

### Занятие 2:

- Word Embeddings (Word2Vec, Navec)
- Классификация тональности
- Взгляд архитектора ИИ

## Единый датасет

Все демо — на реальных отзывах RuReviews, один и тот же train/test сплит от начала до конца

# Зачем архитектору ИИ понимать NLP

## Natural Language Processing (NLP)

Обработка естественного языка — область ИИ, которая занимается взаимодействием между компьютерами и человеческим языком

### Примеры применения:

- Google Translate
- Анализ тональности отзывов
- Автодополнение текста
- Детектирование спама в почте

### Основные задачи NLP:

- Классификация текстов
- Машинный перевод
- Генерация текста
- Анализ тональности

### Взгляд архитектора

Пайплайн предобработки — это не разовый скрипт, а код, который придётся деплоить, масштабировать и обслуживать в проде

# Вызовы и сложности NLP

## 1. Неоднозначность и омонимы:

- **коса** — инструмент / причёска / берег
- **лук** — овощ / оружие
- **ключ** — от двери / источник воды / муз. ключ
- **замок** — дверной / средневековый

## 2. Контекстная зависимость:

- "Я купил новую **мышь**" — животное или устройство?
- "Она посмотрела на **замок**" — укрепление или дверной механизм?
- "Он открыл **счёт**" — в банке или в футбольном матче?

## 3. Морфологическое богатство:

- стол → столом, столы, столов...
- бежать → бегу, бежишь, бежал...
- красивый → красивая, красивые...

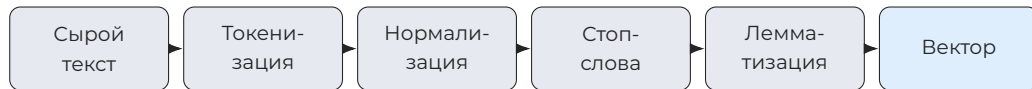
## 4. Идиомы и фразеологизмы:

- "бить баклуши" (бездельничать)
- "как рыба в воде" (комфортно)
- "водить за нос" (обманывать)

## 5. Проблемы с буквой Ё:

- "все" vs "всё"
- "узнает" vs "узнаёт"

## Пайплайн предобработки текста



- **Нормализация:** нижний регистр, удаление пунктуации
- **Стоп-слова:** высокочастотные малоинформативные слова ("и", "в", "на")
- **Лемматизация:** приведение к словарной форме ("бегу" → "бежать") — для русского обязательна из-за богатой морфологии

# Токенизация текста

## Задача

Разбить реальный отзыв (RuReviews) на токены и убрать стоп-слова

```
from natasha import Segmenter, Doc
from nltk.corpus import stopwords
segmenter = Segmenter()
text = "Товар соответствует цене. На 44 размер, рост 160 см. оказались малы."
doc = Doc(text)
doc.segment(segmenter)
tokens = [t.text for t in doc.tokens]
# Товар['', соответствует'', цене'', '.', На'', '44', размер'', ...]
stop_words_ru = set(stopwords.words('russian'))
filtered = [t.lower() for t in tokens
            if t.isalpha() and t.lower() not in stop_words_ru]
# товар['', соответствует'', цене'', размер'', рост'', см'', оказались'', малы']
```

**Проблемы naïve-токенизации:** "Санкт-Петербург", "COVID-19", "user@mail.ru" — Natasha справляется лучше, чем разбиение по пробелам

# Стемминг vs Лемматизация

## Стемминг (Stemming)

- Грубое отсечение окончаний
- Быстрый, но неточный
- Алгоритм: SnowballStemmer

```
from nltk.stem import SnowballStemmer

stemmer = SnowballStemmer('russian')
stemmer.stem('оказались')      # оказа''
stemmer.stem('соответствует')  # соответств''
stemmer.stem('матери')         # матер''
```

## Лемматизация (Lemmatization)

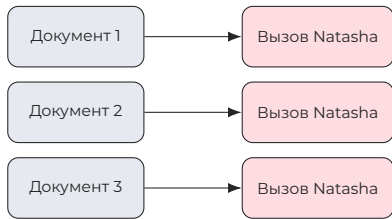
- Морфологический анализ
- Точный, чуть медленнее
- Библиотека: Natasha

```
# morph_tagger, morph_vocab — см. слайд 6
doc = Doc('оказались соответствует матери')
doc.segment(segmenter)
doc.tag_morph(morph_tagger)
for t in doc.tokens:
    t.lemmatize(morph_vocab)
    print(t.text, '->', t.lemma)
# оказались -> оказаться
# соответствует -> соответствовать
# матери -> мать
```

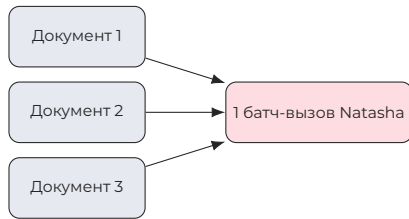
## Рекомендация

Для русского языка с богатой морфологией: **лемматизация**. «матер» (стемминг) — бессмысленный обрубок, «мать» (лемма) — корректная словарная форма. Natasha дополнительно умеет синтаксис и NER — пригодятся в продакшене

## Архитектор: стоимость предобработки на масштабе



По одному документу  
3 вызова



Батч документов  
1 вызов

- Загрузка модели (эмбединги Natasha) — самая дорогая операция; делать её на каждый запрос нельзя
- Вызов "по одному документу" — накладные расходы на каждый запрос; батчинг амортизирует их
- В проде: держать модель загруженной в памяти сервиса, обрабатывать документы батчами, а не поштучно

# Bag of Words (BoW)

## Концепция

Представление документа как "мешка слов" — подсчёт частоты каждого слова

### Пример (реальные отзывы RuReviews):

- Документ 1: "Ужасное качество товара!"
- Документ 2: "Качество не хорошее"

**Словарь:** [качество, не, товара, ужасное, хорошее]

|       | качество | не | товара | ужасное | хорошее |
|-------|----------|----|--------|---------|---------|
| Док 1 | 1        | 0  | 1      | 1       | 0       |
| Док 2 | 1        | 1  | 0      | 0       | 1       |

### Преимущества:

- Простота
- Интерпретируемость

### Недостатки:

- Игнорирует порядок слов
- Высокая размерность
- Не учитывает семантику

## TF-IDF: Term Frequency - Inverse Document Frequency

$$TF(t, d) = \frac{\text{count}(t, d)}{|d|} \quad IDF(t) = \log \frac{N}{df(t)}$$

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

**Интерпретация:** слово частое в документе, но редкое в корпусе → **высокий вес**; слово, встречающееся везде → **низкий вес**

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = ["Ужасное качество товара",
          "Качество не хорошее",
          "Хорошее качество и быстрая доставка"]
vec = TfidfVectorizer()
m = vec.fit_transform(corpus)
print(vec.get_feature_names_out())
# быстрая '' доставка'' качество'' не'' товара'' ужасное'' хорошее''
print(m.toarray().round(3))
# [[0.    0.    0.385 0.    0.652 0.652 0.    ]
#   [0.    0.    0.425 0.72 0.    0.    0.548]
#   [0.584 0.584 0.345 0.    0.    0.    0.445]]
```

### Реальный результат

"качество" встречается во всех 3 документах → вес 0.35–0.43 в каждом; "товара"/"ужасное" (только в 1 документе) → вес 0.65

# N-граммы (N-grams)

Последовательности из N слов, сохраняющие локальный контекст. **Пример:** "Ужасное качество товара"

- **Unigrams:** ужасное, качество, товара
- **Bigrams:** ужасное качество, качество товара
- **Trigrams:** ужасное качество товара

```
from sklearn.feature_extraction.text import CountVectorizer
# Тот же корпус из 3 отзывов, что и на слайде TF-IDF
vectorizer = CountVectorizer(ngram_range=(1, 2))
matrix = vectorizer.fit_transform(corpus)
print(sorted(vectorizer.get_feature_names_out()))
# быстрая'', быстрая' доставка', доставка'', качество'', ...]
print(len(vectorizer.get_feature_names_out())) # 14 признаков на 3 документа
```

**Недостаток:** 14 признаков на 3 коротких отзывах — на реальном корпусе размерность быстро непрактична

## Сравнение методов векторизации

| Метод    | Размерность      | Семантика | Порядок  | Скорость |
|----------|------------------|-----------|----------|----------|
| BoW      | Высокая          | Нет       | Нет      | Быстро   |
| TF-IDF   | Высокая          | Частично  | Нет      | Быстро   |
| N-grams  | Очень высокая    | Частично  | Локально | Быстро   |
| Word2Vec | Низкая (100-300) | Да        | Нет      | Средне   |

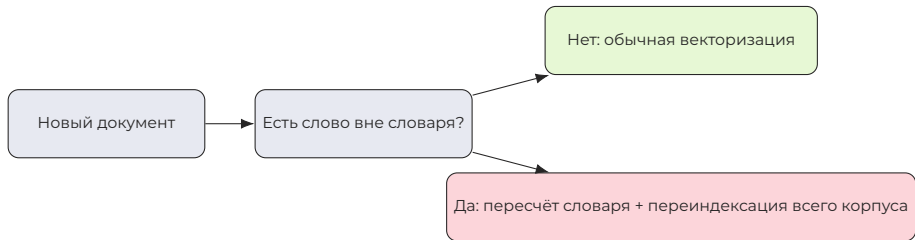
### Классические методы:

- Sparse vectors (разреженные)
- Размерность = размер словаря
- Не захватывают семантику
- Хороши для baseline

### Word Embeddings:

- Dense vectors (плотные)
- Низкая размерность
- Семантическая близость
- Лучшее качество

## Архитектор: словарь векторизатора в проде



- У BoW/TF-IDF словарь фиксируется на 'fit()' — новое слово вне словаря просто игнорируется молча
- Чтобы учесть новые слова — нужен полный 'fit()' заново на всём корпусе и пересчёт всех сохранённых векторов
- В проде это недёшево: переобучение векторизатора обычно требует переиндексации всего хранилища эмбедингов/векторов

# Итоги занятия 1

## Что сделали

Векторизовали текст числами: BoW, TF-IDF, N-граммы — все **sparse**, ни один не видит смысла слов

## Вопрос на следующее занятие

"Кошка" и "кот" для BoW/TF-IDF — два никак не связанных признака. Как получить представление, где похожие по смыслу слова оказываются рядом?

# Спасибо за внимание!

Вопросы?

Сегодня — shallow embeddings (Word2Vec, Navec). В семинаре 7.11 — по-настоящему глубокие архитектуры: RNN, LSTM, Attention, BERT

Контакты: [https://t.me/m\\_polyak](https://t.me/m_polyak)